FULL STACK AI ENGINEERING / CAPSTONE BLUEPRINT

QueryTrust AI

A Reliable Natural-Language Analytics Workbench

A detailed product, architecture, security, evaluation, and implementation specification for Google Antigravity.

PROJECT TYPE	Full Stack AI Engineering college capstone
BUILD WINDOW	7-day MVP implementation
DOMAIN	Synthetic network-device assets and warranty analytics
STACK	React + TypeScript, FastAPI, PostgreSQL, scikit-learn, Docker
LLM MODE	Configurable real provider + clearly labelled demo fallback
SPEC VERSION	1.0 - 14 October 2026

BUILD PHILOSOPHY

Make one end-to-end path work, measure it, secure it, test it, and document its limits. Do not trade correctness for a longer feature list.

Synthetic educational data only. Not an official Cisco product and not a replacement for Snowflake or ThoughtSpot.

Document map

Use the Markdown source for direct agent ingestion. This formatted specification is the human-readable contract for scope, safety, testing and course alignment.

1. Executive brief	2. Product positioning and differentiation
3. Scope: build this first, not everything at once	4. Users, roles, and permissions
5. Core user journeys	6. UX and page specification
7. Technical architecture	8. Database model and synthetic dataset
9. Natural-language analytics pipeline	10. SQL safety and execution contract
11. REST API contract	12. Data-quality subsystem
13. ML component: telemetry anomaly detection	14. Evaluation and regression benchmark
15. Authentication, authorization, and privacy	16. API, performance, and maintainability requirements
17. Suggested repository structure	18. Configuration and local setup
19. Docker and deployment	20. Testing requirements
21. Definition of done / acceptance criteria	22. Seven-day build plan
23. Suggested live demo (5-7 minutes)	24. Viva preparation: understand these decisions
25. Required documentation and hand-in package	26. Antigravity implementation protocol
27. Course alignment checklist	28. Risks and trade-offs
29. Reference links for implementation	30. Final build principle

Read this first

This file is the product and engineering source of truth for the project. The developer agent must follow it unless the student explicitly approves a change. The companion ANTIGRAVITY_MASTER_PROMPT.md explains how to give this specification to Google Antigravity. The root-level AGENTS.md contains persistent implementation rules.

How to use this package

1. Create or open a new, empty workspace in Antigravity.
2. Copy PROJECT_SPEC.md, ANTIGRAVITY_MASTER_PROMPT.md, and AGENTS.md into the workspace root.
3. Paste the master prompt into Antigravity and ask it to begin with Milestone 0 only.
4. Review the proposed plan, then let it implement one milestone at a time.
5. Do not accept a claim that a milestone is complete until its tests and visual checks have actually run.

Evidence and assumptions

- The networking-device example is a synthetic educational scenario inspired by the professor's recording. It is not actual Cisco customer data. Do not use Cisco logos, confidential files, or imply that this is an official Cisco product.
- The project handout calls for REST APIs, SQL/NoSQL design knowledge, authentication and authorization, React/Vue interfaces, dashboards, model serving, Docker/Kubernetes, and model monitoring/versioning. This project prioritizes a working React/FastAPI/PostgreSQL application, role-based access control, a small ML inference endpoint, Docker Compose, and measurable evaluation. Kubernetes and advanced observability are stretch work unless the instructor specifically requires them for this checkpoint.
- Existing products already provide natural-language analytics and enterprise governance. The project must not claim to invent text-to-SQL, semantic layers, or query evaluation. Its educational contribution is a small, inspectable implementation that combines safe read-only execution, explicit ambiguity handling, multi-turn regression evaluation, data-quality profiling, and transparent measurement.
- No model or evaluation accuracy may be fabricated. Report only metrics produced by a real run and describe the dataset, test cases, model/provider, and limitations.

1. Executive brief

1.1 One-sentence description

QueryTrust AI is a full-stack natural-language analytics workbench that lets users ask questions about a structured networking-asset database, previews and validates AI-generated SQL before controlled read-only execution, profiles data quality, flags unusual telemetry, and measures answer reliability with a repeatable regression benchmark.

1.2 The problem

Business users often have data in SQL tables but do not know SQL, database schemas, or analytics methods. A language model can translate questions into SQL, but generated queries may be invalid, use the wrong business definition, mishandle follow-up questions, or request unsafe operations. A useful system needs more than a chat box: it needs schema context, governed metric definitions, controlled execution, useful visualizations, logs, and a way to test whether changes make answers better or worse.

1.3 The proposed solution

Build a local-first, full-stack prototype with these connected capabilities:

1. A dashboard over synthetic networking assets.
2. A natural-language question interface.
3. SQL preview, structural validation, and read-only execution.
4. A clarification workflow for ambiguous or unsupported requests.
5. Conversation history and basic multi-turn context.
6. Data-quality profiling for missing, inconsistent, and potentially duplicated values.
7. A small anomaly-detection endpoint for synthetic device telemetry.
8. A benchmark runner with verified questions, expected results, and regression reporting.
9. User authentication and role-based permissions.
10. Containerized local deployment, health checks, structured logs, and an implementation README.

1.4 What success looks like

A reviewer can launch the app, sign in using a documented local demo account, inspect the sample data, ask a question in plain English, review the generated SQL, execute it safely, see the result in a table or chart, inspect a data-quality report, run an anomaly scan, and launch an evaluation suite. The README explains how to configure an LLM provider or run clearly labelled demo mode.

2. Product positioning and differentiation

2.1 Do not recreate an entire enterprise BI product

Snowflake Cortex Analyst and ThoughtSpot Spotter already offer natural-language analytics and governed-data functionality. Snowflake documents a verified-query evaluation workflow and notes that its documented evaluation feature processes test questions independently rather than evaluating multi-turn conversations in a single evaluation run. ThoughtSpot describes semantic definitions, traceability, and governed analytics. These capabilities establish that the general category is real; they also mean this student project must be honest about its scope.

2.2 The project's focused contribution

Position QueryTrust as an **educational reliability and governance prototype**, with four areas to demonstrate:

- **Preview-before-execute:** show the proposed SQL and validation findings before a user runs it.
- **Ambiguity-aware conversations:** ask a question when a metric or phrase such as “high risk” has no defined meaning, rather than inventing business logic.

- **Multi-turn regression benchmark:** test follow-up questions such as “filter to the North region,” “now group by device type,” and “only show the active ones.” Compare supported multi-turn conversations across code/prompt changes.
- **Visible evidence:** retain query status, execution time, row count, prompt/provider version, and benchmark outcomes without logging secrets or unnecessary result data.

These are the project's chosen design goals, not a claim that no commercial tool has any comparable feature. The report must compare observed behavior only when a reproducible test has actually been run.

2.3 Proposed evaluation question

“On a curated synthetic benchmark, does adding schema-aware context, ambiguity handling, SQL structural validation, and controlled read-only execution improve answer correctness and policy enforcement compared with a simple LLM-to-SQL baseline?”

The answer must come from recorded experiments, not assumptions. A guarded system may be safer but still make mistakes; measure the result and describe remaining failures.

3. Scope: build this first, not everything at once

3.1 P0 - required for the MVP

1. React + TypeScript + Vite frontend with a polished responsive layout.
2. FastAPI backend with versioned /api/v1 endpoints and OpenAPI docs.
3. PostgreSQL schema, migrations, deterministic synthetic data generator, and seed command.
4. Login and role-based access control.
5. Dashboard with device, warranty, site, and service-event summaries.
6. Natural-language-to-SQL workflow using a configurable LLM provider.
7. Clearly labelled deterministic demo mode when no external API key is configured.
8. SQL preview, parser-based validation, table allowlist, query limits, timeout, and read-only database execution.
9. Clarification/unsupported intent handling and persisted conversation messages.
10. Data-quality profiling and a findings report.
11. A small telemetry anomaly-detection model served through FastAPI.
12. A benchmark of at least 30 manually reviewed test cases, including multi-turn and policy-rejection cases.
13. Automated backend tests and frontend build/type checks.
14. Docker Compose startup path and complete setup documentation.
15. Basic health checks, structured application logs, query history, and prompt/provider version metadata.

3.2 P1 - implement after P0 passes

- Redis caching for dashboard summary responses with a short TTL and safe cache keys.
- Rate limiting for authentication and query-generation endpoints.
- Admin-only audit-log view and evaluation-run page.
- More polished multi-turn filter-state handling.
- A one-click benchmark run and baseline-versus-guarded comparison report.
- ML model version metadata and a reproducible model-fit command.
- Accessibility, responsive layout, and UX refinements based on browser testing.

3.3 Stretch only - do not let these block the MVP

- Kubernetes manifests and a local kind/minikube deployment.
- MLflow experiment tracking.
- Prometheus/Grafana dashboards.
- Multiple LLM agents with separate orchestration.

- pgvector or other semantic retrieval. Only add it if a demonstrated semantic-search use case exists; SQL aggregation does not need vectorization.
- Database replicas or actual write operations.
- Multi-tenant enterprise isolation, SSO, billing, or cloud production hardening.

3.4 Explicit non-goals

- Do not attempt to ingest or vectorize a real 100 GB enterprise database in seven days.
- Do not connect to any real Cisco, university, or employer database.
- Do not permit natural-language requests to change or delete business data.
- Do not fine-tune a foundation model unless the MVP is complete and experiments justify it.
- Do not claim predicted device failure probabilities. The ML component flags statistical outliers only.
- Do not claim production-grade enterprise security based solely on a college prototype.
- Do not create fake charts, fake result rows, fake evaluation numbers, or buttons that silently do nothing.

4. Users, roles, and permissions

The application is single-tenant and uses synthetic data for the MVP.

Role	Allowed actions	Disallowed actions
Viewer	Sign in; view dashboard and device tables; ask approved read-only questions; inspect own conversation/history	Manage users; access admin audit data; execute database writes
Analyst	All Viewer capabilities; run data-quality profiling; run anomaly scans; submit answer feedback	Manage users/roles; execute database writes; bypass query validation
Admin	All Analyst capabilities; manage user roles/status; view audit/query metadata; run benchmark evaluations	Execute write SQL through the natural-language interface; disable the read-only guard

Important invariants:

- The natural-language query tool is read-only for every role, including Admin.
- API authorization must be checked on the backend. Hiding a button in React is not authorization.
- A client-submitted role, SQL string, user ID, or query approval is never trusted as proof of permission.
- The server executes only a previously stored, server-validated query preview identified by an opaque run ID. The client cannot edit the SQL after validation and then execute the modified text.
- The seeded local demo accounts and passwords are for local development only. All demo passwords must be configurable through environment variables and must be changed for any non-local deployment.

5. Core user journeys

5.1 First launch

1. Start the application using the documented setup or Docker Compose command.
2. Health checks confirm that the API and database are ready.
3. The seed script creates synthetic sites, devices, service events, telemetry, and optional development-only demo accounts.
4. A visible “Synthetic Demo Data” label appears in the application.
5. The user signs in and lands on the dashboard.

5.2 Ask a data question

1. User opens **Ask Data** and enters a natural-language question.
2. Backend loads the approved schema and semantic catalog, plus a bounded amount of relevant conversation context.

3. LLM returns a structured intent: query ready, clarification required, or unsupported.
4. If clarification is required, show the question and do not create an executable preview.
5. If ready, parse and validate SQL, enforce table/function allowlists, cap result size, and store a preview record.
6. UI shows the interpreted question, SQL preview, validation status, and assumptions.
7. User clicks **Execute query**.
8. Backend rechecks authorization and preview state, executes the stored SQL in a read-only transaction using a read-only DB role, applies a statement timeout, and enforces row/output limits.
9. UI shows a result table, optional chart, elapsed time, row count, and concise explanation grounded in returned data.
10. User can mark the answer helpful/not helpful and optionally explain the problem.

5.3 Multi-turn conversation

Example:

- User: “Show active devices in the North region.”
- User: “Now group those by device type.”
- User: “Only those with warranties expiring in November 2026.”

The system must preserve relevant prior constraints when interpreting a follow-up. The conversation context must be limited to the current authenticated user's session. If a follow-up cannot be resolved safely, ask a clarification question. Never blindly append the previous SQL or trust it without revalidation.

5.4 Data quality review

1. Analyst opens **Data Quality**.
2. The backend profiles approved business tables.
3. Findings are grouped by severity and type: missing field, invalid date relationship, inconsistent category, possible duplicate, and unexpected status value.
4. The report shows affected column, count, percentage, sample identifiers where safe, and suggested remediation.
5. Suggestions never alter rows automatically. The MVP is a read-only diagnostics tool.

5.5 Anomaly review

1. Analyst opens **Anomaly Monitor**.
2. The backend aggregates numeric telemetry by device and applies a small unsupervised anomaly detector.
3. The page shows the flagged observations, feature values, anomaly score/rank, and model version.
4. UI clearly states: “Anomaly score is not a probability of failure; this is a synthetic-data demonstration.”

5.6 Evaluation run

1. Admin opens **Evaluation Lab**.
2. Selects the curated benchmark suite.
3. The backend executes the benchmark using deterministic/mock mode for unit tests or a configured real LLM provider for live generation tests.
4. It compares actual results/expected behavior, records duration, and shows pass rate by category.
5. Reports state the number of tests, exclusions, provider/model, dataset version, and whether the results came from real inference or demo fixtures.
6. No accuracy metric is hard-coded or faked.

6. UX and page specification

Use a clean, enterprise-style interface: light neutral content surfaces, navy/charcoal navigation, restrained indigo or teal accents, clear typography, strong contrast, consistent spacing, and purposeful charts. Do not overuse gradients,

animations, or decorative cards. Use one consistent design system. Responsive layout must work at laptop and mobile widths.

6.1 Global shell

- Left navigation: Overview, Ask Data, Data Quality, Anomaly Monitor, Query History, Evaluation Lab (Admin), Administration (Admin).
- Header: page title, environment badge (“DEMO” or “LLM CONNECTED”), synthetic-data badge, user menu.
- Common UI states: loading skeleton, empty state with next step, recoverable error with retry, success toast, and confirmation for sensitive admin actions.
- Never show secret keys in UI or logs.

6.2 Login

- Email and password form, validation, show/hide password, loading and error states.
- Do not expose whether an arbitrary account exists through overly detailed auth errors.
- Provide a clearly labelled local-demo credential hint in development mode only.

6.3 Overview dashboard

KPI cards:

- Total devices.
- Active devices.
- Warranties expiring in the next 30 days.
- Open service events.
- Data-quality findings from the latest run.

Charts/tables:

- Devices by type/status.
- Warranty-expiry trend by date bucket.
- Open service events by severity/status.
- Sites with the most upcoming warranty expirations.
- Recent query activity (metadata only).

All numbers must come from API/database results. No hard-coded dummy KPI values in the finished application.

6.4 Ask Data

- Conversation list or recent sessions.
- Chat-style question input, send button, example question chips.
- Visible current mode (configured provider vs deterministic demo mode).
- For a ready query: expandable SQL preview, validation checklist, table allowlist, warnings/assumptions, Execute button.
- For clarification: display a clear follow-up field and do not show Execute.
- For blocked query: explain the policy in simple language; do not expose internal secrets or stack traces.
- Result display: table with pagination if necessary, column types where appropriate, downloadable CSV of the displayed/approved result only if implemented safely, optional chart for suitable aggregations, and concise evidence-based explanation.
- Limit the number of rows returned by a query. If more results are available, tell the user and recommend a narrower filter.

6.5 Data Quality

- Overall data-quality score may be displayed only if its formula is defined and shown. Prefer counts and percentages to a vague score.
- Finding table: severity, table, column, rule, affected rows, percentage, example IDs, suggested action.
- Filters by severity/table/rule.
- “Run profile” triggers a real backend job and displays completion time.

6.6 Anomaly Monitor

- Run scan button.
- Metric definitions and model version.
- Table of anomalies with device ID, site, recorded time, selected telemetry values, score/rank, and explanation that this is an outlier flag.
- Show model status and last-fit timestamp.

6.7 Query History

- Current user's query history by default; Admin may inspect metadata for all users.
- Status, timestamp, question, execution duration, returned row count, provider/model, prompt version, validation outcome.
- Do not log or expose password/token/secret values.

6.8 Evaluation Lab

- Admin-only page.
- Suite count, run button, real run timestamp, provider/mode, pass rate, latency summary, results by category.
- Baseline-versus-guarded comparison only when both sets actually ran on the same benchmark/data version.
- Failed-case details should include test ID, failure category, expected behavior, observed outcome, and safe diagnostic detail.

6.9 Administration

- Admin-only user listing, role and active status changes.
- Confirm role changes and status changes.
- Do not provide any feature to disable SQL validation from the UI.

7. Technical architecture

7.1 Technology choices

Layer	Required choice	Notes
Frontend	React + TypeScript + Vite	Use typed API client and reusable components
UI styling	Tailwind CSS or a similarly consistent component system	Choose one approach and keep the bundle consistent
Charts	Recharts or Plotly	Use real API data
Backend	Python 3.12+ + FastAPI	Versioned REST APIs, Pydantic request/response models
Database	PostgreSQL	Main persistent database
ORM/migrations	SQLAlchemy 2.x + Alembic	No uncontrolled schema creation in production startup
SQL parser	SQLGlot	Parse in PostgreSQL dialect before execution
Analytics	Pandas + NumPy	Profiling and metric preparation
ML	scikit-learn IsolationForest	Unsupervised telemetry-outlier demonstration; not failure prediction
Authentication	Password hashing + signed access tokens	Use backend authorization checks and role scopes
Optional cache	Redis	Cache dashboard aggregates only after core correctness is proven
Tests	pytest + HTTPX/TestClient; frontend build/type checks	No live LLM dependency for unit tests
Deployment	Docker + Docker Compose	Core deliverable; Kubernetes is stretch/requirement-dependent

Use maintained, mutually compatible versions available in the environment. Pin the resolved dependencies once the application runs. Do not invent package versions. Do not use obsolete or abandoned authentication libraries just to save time.

7.2 Component responsibilities

- **Frontend:** navigation, authentication UI, form state, previews, result tables/charts, accessible loading/error states.
- **FastAPI routes:** input validation, authentication dependency, authorization, response schemas, status codes.
- **Application services:** analytics orchestration, data profiling, anomaly scoring, evaluation runs, audit metadata.
- **LLM adapter:** calls a configured provider with a strict prompt/output schema; isolates provider-specific code.
- **SQL guard:** parser-based statement validation, table allowlist, blocked functions, limit enforcement, and safety report.
- **Read-only query executor:** executes only a stored validated preview using a read-only DB account and bounded execution settings.
- **PostgreSQL:** stores business data, users, conversation/query metadata, feedback and evaluation summaries.
- **Seed utility:** deterministic synthetic data with controlled data-quality issues and telemetry outliers.

7.3 Environment modes

Support these modes:

- LLM_PROVIDER=demo: deterministic, clearly labelled fallback for supported example intents. It must never claim that an external model was called.
- LLM_PROVIDER=gemini: use a real Google Gemini API only when a separate valid runtime API key is configured.

- LLM_PROVIDER=openai_compatible: use the OpenAI SDK and configured compatible endpoint/key/model.

Use a provider interface so other APIs could be added later. Do not assume the credentials used by the Antigravity coding assistant automatically exist in the running application. If no key is present, the app must still start in demo mode and explain its limitations.

Never hard-code an API key. Provide .env.example, load secrets from environment variables, ignore .env in git, redact secrets in logs, and fail with a helpful message for invalid provider configuration.

8. Database model and synthetic dataset

8.1 Required tables

`users`

- id UUID primary key.
- email unique, normalized to lowercase.
- display_name.
- password_hash.
- role constrained to admin, analyst, viewer.
- is_active boolean.
- created_at, updated_at.

`sites`

- id integer primary key.
- site_code unique.
- site_name.
- city.
- region.

`devices`

- id integer primary key.
- asset_tag unique and non-null.
- device_type text; intentionally allow a small number of inconsistent case/category variants for profiling.
- vendor, model, firmware_version.
- site_id foreign key to sites.
- status text, with a small number of casing/whitespace inconsistencies.
- installed_at date.
- warranty_expiry nullable date.
- serial_number nullable text; controlled repeated values may be inserted to test potential-duplicate detection.
- created_at, updated_at.

`service_events`

- id integer primary key.
- device_id foreign key to devices.
- event_type, severity, status.
- opened_at, nullable closed_at.
- description synthetic and non-sensitive.

`telemetry_readings`

- id integer primary key.
- device_id foreign key to devices.
- recorded_at timestamp.
- cpu_percent, memory_percent, temperature_c, packet_loss_percent, latency_ms numeric fields.
- Optional known_injected_outlier boolean for offline evaluation only; do not expose this label to the inference feature as an input.

`chat_sessions` and `chat_messages`

Persist conversation owner, title, timestamp, role, message content and basic session state. Enforce that non-admin users can access only their own sessions.

`query_runs`

Persist question, session/user ID, generated SQL, status, validation findings JSON, provider/model, prompt version, row count, execution duration, error category, created/executed timestamps, and benchmark linkage where relevant. Do not persist large result sets by default.

`query_feedback`

Persist query run ID, user ID, helpful/not-helpful flag, optional explanation, timestamp.

`data_quality_runs` and `data_quality_findings`

Persist run metadata and structured findings so the UI can display the most recent report.

`evaluation_runs`

Persist benchmark name/version, provider/mode, start/end times, totals, category metrics, failures summary, and metadata as JSONB.

`ml_model_versions`

Persist model name, version, feature list, fit time, dataset/version identifier, configuration and selected evaluation metrics. The MVP may serialize this as JSON/DB metadata rather than introduce a full model registry.

8.2 Synthetic data generator

Create a deterministic script such as python -m app.seed.seed_database.

Initial target sizes:

- 40-60 sites.
- 5,000 devices.
- 5,000-12,000 service events.
- 20,000-50,000 telemetry readings.

Generate plausible but fake site names, locations, device types, installation dates, warranty dates, incidents, and telemetry. Use a fixed random seed for reproducibility. Include controlled quality problems: missing warranty dates, inconsistent capitalization, unexpected status values, repeated serial numbers, warranty date earlier than installation date, and telemetry outliers. Document exactly which issues are injected and at what approximate rates.

Do not require internet access to seed the dataset. Do not download company data. The UI must label the dataset synthetic.

8.3 Semantic catalog

Create a central semantic catalog in code or YAML and use it in LLM context and evaluation. Include allowed tables, column descriptions, join keys, metric definitions, synonyms and example questions.

Required business definitions:

- **Device count:** count distinct devices.id.
- **Active device:** lower(trim(status)) = 'active'.
- **Open service event:** normalized status is not resolved, closed, or cancelled.
- **Warranty expiring in N days:** warranty date is on/after the chosen reference date and before reference date + N days; document the inclusive/exclusive boundary.
- **Site/region:** sourced through devices.site_id -> sites.id.
- **High risk:** undefined unless the user explicitly chooses a documented rule, such as imminent warranty expiry, repeated incidents, or anomalous telemetry. Ask a clarification question otherwise.
- **Revenue/profit/customer churn:** unsupported in this dataset; do not fabricate SQL or answers for these concepts.

9. Natural-language analytics pipeline

9.1 Structured LLM response

Ask the provider for strict JSON matching a Pydantic model similar to:

```
{
  "intent_status": "ready | clarification_needed | unsupported",
  "interpreted_question": "Plain language interpretation",
  "clarification_question": null,
  "sql": "SELECT ...",
  "assumptions": ["..."],
  "business_terms_used": ["active device"]
}
```

The application must validate the returned shape. Reject malformed JSON and retry at most once with a repair instruction. If it still fails, return a recoverable error. Never execute a free-form natural-language answer as SQL.

9.2 Prompt requirements

The server-side prompt must:

- Say the model generates a candidate query only; the model cannot approve or execute its own query.
- Include the approved schema and semantic definitions, not the whole dataset.
- Include the user's question and bounded conversation history.
- Tell the model to ask a clarification when required information/business definitions are ambiguous.
- Tell the model to return unsupported when requested information is not in the schema.
- Require exactly one read-only PostgreSQL SELECT query (CTEs that remain read-only SELECTs may be supported after parsing).
- Forbid INSERT, UPDATE, DELETE, MERGE, DROP, ALTER, CREATE, TRUNCATE, GRANT, REVOKE, COPY, CALL, transaction control and multiple statements.
- Forbid system catalogs and tables outside the allowlist.
- Ask for only columns needed to answer the question.
- Never claim the generated query is safe merely because it follows these instructions.

9.3 Preview is not execution

preview must not execute the generated SQL against business tables. It parses, validates, stores and returns the candidate query. The user then chooses Execute. The execute endpoint accepts a stored preview ID, not arbitrary SQL text.

9.4 Explain the result honestly

The result summary may mention only facts supported by returned rows/aggregates. It should report the executed query's relevant filters and row count. It must not imply causal insight from a simple correlation or state a failure probability from an anomaly score. If the result is empty, say no rows matched the specified filters.

10. SQL safety and execution contract

This section is a release blocker. A prompt alone is never a security boundary.

10.1 Multiple independent controls

1. Parse generated SQL with SQLGlot using the PostgreSQL dialect.
2. Require exactly one statement.
3. Require the root statement and nested CTEs/subqueries to be read-only SELECT operations.
4. Allow only approved business tables and approved columns; reject system catalogs and unknown relations.

5. Reject write/DDL/privilege/administrative commands and risky functions such as file access, sleeping, configuration mutation, external database connections, or unapproved dynamic SQL.
6. Add or clamp a maximum row limit (initial default 200; hard maximum 500 for MVP).
7. Apply statement timeout (initial target 3-5 seconds) and bounded response size.
8. Execute through a separate database user with SELECT privileges only on approved views/tables.
9. Run inside a read-only transaction where supported.
10. Persist status and validation metadata; do not leak stack traces or database credentials to the client.
11. If any control fails or raises uncertainty, block the query and explain the reason.

10.2 Allowlist

Only allow the specific application tables needed for analytics: devices, sites, service_events, and approved read-only telemetry views/tables. Do not expose users, token/session internals, chat logs, query feedback, audit records, evaluation metadata, or pg_catalog to the LLM query executor.

10.3 Execution rules

- The LLM cannot choose a database URL or connection role.
- The browser cannot submit a new SQL string to the execute endpoint.
- The database reader cannot write, create functions, access external files or modify schema.
- Validate again at execution time; don't trust preview state alone.
- Enforce cancellation/timeout where possible.
- Return only bounded rows and safe column metadata.
- Do not execute unsafe queries “to see what happens.” Use unit tests to test rejection.
- Tests must include attempts using semicolons, comments, CTEs, nested subqueries, schema-qualified tables, forbidden functions, SELECT INTO, system catalog access, and all common write/DDL commands. Check both parser behavior and actual DB permissions.

11. REST API contract

All business endpoints are under /api/v1. Use Pydantic schemas, typed responses, pagination, consistent errors, and OpenAPI documentation. API names may be refined during implementation only if all frontend, tests, and docs are updated together.

Method	Endpoint	Purpose	Access
GET	/health	Process liveness	Public, no secrets
GET	/ready	DB/required-service readiness	Internal status only
POST	/api/v1/auth/login	Authenticate and issue access token	Public, rate limited
POST	/api/v1/auth/refresh	Refresh token if implemented	Authenticated/refresh token
GET	/api/v1/auth/me	Current user profile/role	Authenticated
GET	/api/v1/dashboard/summary	KPI aggregates and chart data	Authenticated
GET	/api/v1/devices	Filtered/paginated device listing	Authenticated
GET	/api/v1/sites	Sites and approved filters	Authenticated
POST	/api/v1/chat/sessions	Create a conversation	Authenticated
GET	/api/v1/chat/sessions	List current user's sessions	Authenticated; admin rules documented
GET	/api/v1/chat/sessions/{id}/messages	Read session messages	Owner or explicitly authorized admin
POST	/api/v1/analytics/preview	Interpret question, generate/validate/store SQL preview	Authenticated

Method	Endpoint	Purpose	Access
POST	/api/v1/analytics/execute	Execute server-stored validated preview	Authenticated, owner, valid state
POST	/api/v1/analytics/feedback	Record user feedback	Authenticated, owner of query
GET	/api/v1/analytics/history	Query history metadata	Own history; admin may audit metadata
GET	/api/v1/data-quality/summary	Latest data-quality summary	Analyst/Admin or approved Viewer read access
POST	/api/v1/data-quality/run	Run profiling checks	Analyst/Admin
GET	/api/v1/anomalies/summary	Latest anomaly summary	Analyst/Admin or approved Viewer read access
POST	/api/v1/anomalies/scan	Run the anomaly detector	Analyst/Admin
GET	/api/v1/evaluations	List evaluation runs	Admin
POST	/api/v1/evaluations/run	Run a benchmark	Admin
GET	/api/v1/admin/users	List users	Admin
PATCH	/api/v1/admin/users/{id}	Change role/active status	Admin; validate role transition
GET	/api/v1/admin/audit-logs	Query security/query metadata	Admin

11.1 Preview response example

```
{
  "query_run_id": "opaque-server-generated-id",
  "status": "previewed",
  "intent_status": "ready",
  "interpreted_question": "Find active routers in the North region.",
  "sql_preview": "SELECT ...",
  "validation": {
    "passed": true,
    "checks": ["single_statement", "select_only", "table_allowlist", "row_limit"],
    "warnings": []
  },
  "assumptions": ["Active status is matched case-insensitively."],
  "can_execute": true
}
```

This is a contract example, not the output of a run. The implementation must use a real UUID/opaque ID and real validation results.

11.2 Error shape

Use a stable response shape, for example:

```
{
  "error": {
    "code": "QUERY_BLOCKED",
    "message": "This request cannot be executed because it is outside the read-only analytics policy.",
    "request_id": "..."
  }
}
```

Do not return raw Python tracebacks, SQL connection strings, API keys, stack traces, password hashes, or internal prompt secrets.

12. Data-quality subsystem

Implement deterministic rules and report counts/percentages with clear definitions.

Required rules:

1. **Missing values:** count null/blank values in warranty_expiry, firmware_version, serial_number, and other selected fields.
2. **Invalid date relationship:** warranty expiry earlier than installation date.
3. **Potential duplicate serial:** repeated normalized non-null serial numbers.
4. **Inconsistent device category:** values that differ only by capitalization/whitespace or violate the approved category list.
5. **Unexpected status:** values outside the approved status vocabulary.
6. **Telemetry range violations:** CPU/memory/packet loss percentages outside valid bounds or implausible values.
7. **Stale telemetry:** device has no reading within a configurable period, if data supports the check.

Every finding must include rule_id, severity, table_name, column_name if relevant, affected_count, total_count, percentage, a brief explanation, recommended next action, and safe sample IDs. Avoid exposing excessive raw values.

A profiling run must store when it ran, the ruleset version, dataset row counts and findings. It must not silently modify the underlying data. Any data-cleaning “fix” is only a recommendation in this MVP.

13. ML component: telemetry anomaly detection

13.1 Purpose

Include one small, genuine ML inference component so the project demonstrates model serving, evaluation metadata, and version tracking. Use scikit-learn IsolationForest on aggregated or record-level synthetic telemetry features such as CPU, memory, temperature, packet loss and latency.

13.2 Requirements

- Provide a reproducible training/fit script or startup fit step clearly separated from request inference.
- Use fixed random seeds for the demo training procedure.
- Save/load a model artifact safely from a known application directory; do not load user-supplied pickle/joblib files.
- Validate and scale/check feature input as needed.
- Expose a FastAPI endpoint for summary/scan operations.
- Track model name, model version, training timestamp, dataset version, feature list, contamination/configuration and evaluation information.
- Show model version in the UI and logs.
- Return score/rank/flag; call it an anomaly score, not a calibrated failure probability.
- Include tests for missing features, invalid numeric values, empty datasets and repeatability.
- If the fitted model artifact is absent, provide an explicit setup error or safe fit process; do not show fake model results.

13.3 Evaluation caveat

Because this uses synthetic data and no verified failure labels, do not report clinical-style or business-grade predictive accuracy. If a known injected-outlier label is retained for the offline benchmark, keep it out of the model input and calculate only appropriate offline detection metrics such as precision/recall at a specified threshold. Clearly identify them as synthetic benchmark metrics.

14. Evaluation and regression benchmark

This is a key differentiator and must be implemented as real code, not just a screen with sample percentages.

14.1 Benchmark composition

Create at least 30 reviewed cases in a versioned JSON/YAML file, ideally 40-60 after the MVP works:

- Simple filters and sorting.
- Counts and group-by aggregations.
- Date ranges with fixed absolute dates for deterministic expected results.
- Joins across devices/sites/service events.
- Semantic terms such as “active” and “open event.”
- Multi-turn conversations with at least 8 follow-up steps/cases.
- Ambiguous questions that should ask for clarification.
- Unsupported questions that should decline safely.
- Security-policy attempts that should be blocked before execution.
- Edge cases: empty result, nulls, repeated statuses, row-limit clamp, unknown column/table.

Every answerable case must have manually reviewed expected SQL or expected result rows/aggregates; multi-turn cases must include the prior turns. Security/ambiguity cases must specify expected behavior. The evaluator should not use the ground-truth expected SQL as context during the model's generation request.

14.2 Metrics and definitions

- **Execution correctness:** a query's normalized result equals the verified expected result for the benchmark case. Compare result sets/aggregates appropriately; exact SQL-text equality is not the primary correctness criterion.
- **Safe rejection rate:** fraction of policy-violating cases correctly blocked.
- **Forbidden execution count:** number of prohibited operations that actually execute; target is zero.
- **Clarification accuracy:** fraction of cases where ambiguous inputs trigger the expected clarification behavior and resolvable inputs do not get needlessly blocked.
- **Multi-turn correctness:** fraction of conversation steps whose result matches the expected result after applying the intended context.
- **Latency:** elapsed time by stage and end-to-end. Report median and p95 only where enough measurements exist.
- **LLM call/error rate:** for real provider runs, record rate-limit/timeouts/schema-parse failures without leaking raw request secrets.

An “exact SQL match” can be a diagnostic metric but should not replace result correctness. Different SQL queries can be equivalent.

14.3 Baseline comparison

Implement a basic baseline generator and the guarded QueryTrust path using the same benchmark and same dataset version. Keep both subject to the same restrictive database-level read-only permissions; never disable essential safety for the baseline. Compare correct answers, blocked unsafe cases, clarification behavior, multi-turn performance, and latency. Document the exact differences between implementations.

14.4 Reproducibility

Each evaluation run records benchmark version, seed data version, provider/mode, model identifier, prompt version, time, test count, exclusions, failures and metrics. Unit tests use a fake/mock provider so CI passes without internet or API credentials. Real-provider evaluations are opt-in and label their outputs as provider-generated. Do not place invented accuracy numbers in the frontend or README.

15. Authentication, authorization, and privacy

Minimum controls:

- Hash passwords with a maintained password-hashing implementation; never store plaintext passwords.
- Signed short-lived access tokens; validate signature, expiry and active-user status.
- If refresh tokens are implemented, rotate them, persist/revoke them appropriately, and avoid logging them. If not implemented in the MVP, document the limitation rather than pretending it exists.
- Enforce RBAC dependencies on backend routes.
- Rate-limit login and expensive LLM endpoints.
- Restrict CORS to the frontend origin in configured environments.
- Validate all request parameters and pagination bounds.
- Store secrets in environment variables. Commit only .env.example placeholders.
- Mask secrets and avoid logging raw sensitive question data unless needed for the educational synthetic dataset; prefer query IDs and metadata.
- Use parameterized statements for application queries. For LLM-generated analytics SQL, parse and restrict the AST, use a database read-only role, and bound execution.
- Never permit the LLM endpoint to choose SQLAlchemy connection strings or run operating-system commands.
- Avoid arbitrary file uploads in the MVP.
- Include a visible local-demo warning about demo credentials and synthetic data.

16. API, performance, and maintainability requirements

- Use /api/v1 versioning for business endpoints.
- Validate responses with Pydantic models; do not return raw ORM objects.
- All list endpoints must have validated pagination and maximum page sizes.
- Use database indexes on commonly filtered/joined fields (foreign keys, status, warranty expiry, site region where appropriate).
- Use bounded LLM timeouts and at most one repair retry for malformed output.
- Never block the entire app if the optional LLM provider is unavailable; show a helpful configuration/runtime state and allow demo mode.
- Do not cache user-sensitive responses without a correct authorization-aware cache key. If Redis is implemented, cache only safe aggregates initially, use a short TTL (e.g., 60 seconds), and show cache behavior in logs/metrics.
- Health and readiness endpoints must not disclose database credentials or internal service details.
- Use structured application logging with request IDs, status, duration, provider, prompt version and query ID. Avoid logging raw result rows.
- Keep dependencies minimal and avoid building a microservice for each feature.

17. Suggested repository structure

Antigravity may make small changes if it documents them, but preserve clear separation of frontend, backend, infrastructure and docs.

```
querytrust-ai/
├── AGENTS.md
├── README.md
├── .env.example
├── .gitignore
├── docker-compose.yml
├── Makefile
├── PROJECT_SPEC.md
├── ANTIGRAVITY_MASTER_PROMPT.md
├── docs/
│   ├── ARCHITECTURE.md
│   ├── API_CONTRACT.md
│   ├── SECURITY.md
│   ├── EVALUATION_PLAN.md
│   ├── DEMO_SCRIPT.md
│   └── BUILD_STATUS.md
├── backend/
│   ├── pyproject.toml
│   ├── alembic.ini
```

```
│   ├── alembic/
│   ├── app/
│   │   ├── main.py
│   │   ├── core/            # settings, auth, logging
│   │   ├── db/              # engine, session, models
│   │   ├── api/v1/          # routers
│   │   ├── schemas/         # Pydantic input/output
│   │   ├── services/        # query, profile, anomaly, eval
│   │   ├── ai/              # provider interface and prompts
│   │   ├── semantic/        # approved schema and definitions
│   │   ├── seed/            # deterministic data generation
│   │   └── evaluation/      # versioned benchmark cases/runner
│   └── tests/
├── frontend/
│   ├── package.json
│   ├── src/
│   │   ├── app/
│   │   ├── components/
│   │   ├── features/
│   │   ├── lib/             # typed API client
│   │   ├── pages/
│   │   └── styles/
│   └── ...
├── scripts/
│   ├── seed.sh
│   └── run_evaluation.sh
└── infra/
    └── k8s/                 # optional stretch only
```

18. Configuration and local setup

Provide .env.example containing placeholders/documented safe defaults such as:

```
APP_ENV=development
APP_NAME=QueryTrust AI
FRONTEND_ORIGIN=http://localhost:5173
DATABASE_URL=postgresql+psycopg://app_user:replace_me@db:5432/querytrust
READONLY_DATABASE_URL=postgresql+psycopg://analytics_reader:replace_me@db:5432/querytrust
JWT_SECRET_KEY=replace-with-a-long-random-secret
ACCESS_TOKEN_MINUTES=30
LLM_PROVIDER=demo
LLM_MODEL=
GEMINI_API_KEY=
OPENAI_API_KEY=
OPENAI_BASE_URL=
LLM_TIMEOUT_SECONDS=20
LLM_MAX_RETRIES=1
QUERY_MAX_ROWS=200
QUERY_HARD_MAX_ROWS=500
QUERY_STATEMENT_TIMEOUT_MS=4000
REDIS_URL=redis://redis:6379/0
CACHE_TTL_SECONDS=60
SEED_SYNTHETIC_DATA=true
SEED_DEMO_USERS=true
DEMO_ADMIN_PASSWORD=change-this-admin-password
DEMO_ANALYST_PASSWORD=change-this-analyst-password
DEMO_VIEWER_PASSWORD=change-this-viewer-password
```

The exact connection strings and variable names must match the application implementation. Do not commit real secrets. Prefer distinct database roles: a migration/seed role and an analytics reader role with SELECT-only grants to approved business tables/views. Compose examples must use local-only credentials and clearly warn against production use.

README should specify:

- prerequisites;

- configuration;
- how to start via Docker Compose;
- how to apply migrations and seed data;
- local demo credentials or how to create them;
- how to connect a real LLM provider;
- how to run unit/integration tests;
- how to run the evaluation benchmark;
- how to stop/reset the app;
- limitations and troubleshooting.

19. Docker and deployment

Required MVP

- Dockerfile for FastAPI backend.
- Dockerfile for React frontend, or a suitable development service plus a documented production build.
- Docker Compose services for PostgreSQL, backend, frontend, and optionally Redis if caching is implemented.
- Persistent database volume for local development.
- Health checks and correct service startup ordering.
- Non-root runtime user where practical.
- No secrets baked into image layers.
- .dockerignore files.
- A command to view logs and a command to stop the stack.

Optional course stretch

If the MVP is already tested and the instructor requires it, add basic Kubernetes manifests for Deployment, Service, ConfigMap/Secret references, liveness/readiness probes, and resource requests/limits. Do not check secret values into YAML. Test the manifests locally if a cluster is available; do not claim they work without verifying them.

20. Testing requirements

Backend unit tests

- Password hashing and token expiry.
- Role permissions and inactive-user rejection.
- Every SQL-guard rule including multi-statement and CTE edge cases.
- Row-limit clamp, allowlisted tables, forbidden functions, no raw SQL accepted by execute API.
- Data-quality counts against a tiny controlled dataset.
- Anomaly model input validation and stable behavior for a fixed dataset.
- Semantic catalog rules and clarification detection.
- Result truncation/output size handling.
- Errors/timeouts from LLM provider.

API integration tests

- Login works and invalid login fails safely.
- Viewer cannot access admin endpoints.
- Users cannot read other users' sessions.
- Preview stores a candidate but does not execute it.
- Only a stored, validated, owner-authorized preview can execute.
- Query reader cannot modify business tables even if a code path has a defect.
- Data-quality and anomaly endpoints return schema-valid results.
- Evaluation run records honest metrics and failure details.

Frontend checks

- TypeScript type-check passes.
- Production build passes.
- Login, dashboard, preview, clarification, blocked query, execution result, data quality, anomalies, and evaluation states render correctly.
- Loading/error/empty states are implemented.
- Browser console has no unhandled errors on core workflows.
- Check mobile layout and keyboard navigation.

Manual smoke checklist

1. Start from a clean database volume.
2. Run migrations and seed data.
3. Sign in as Viewer, Analyst, Admin.
4. Ask a supported question and view SQL preview.
5. Execute and verify a result against SQL directly.
6. Submit an ambiguous question and confirm no query is executed.
7. Trigger a known unsafe SQL test and confirm it is blocked.
8. Run data-quality profiling and anomaly scan.
9. Run benchmark in demo/mock mode.
10. Restart containers and verify persisted data and health checks.

21. Definition of done / acceptance criteria

A milestone or feature is not done just because code has been generated. It is done only when the relevant behavior is implemented and verified.

Application

- [ ] A new user can follow README steps and start the local application.
- [ ] The seeded synthetic dataset loads reproducibly.
- [ ] Login and server-side RBAC work.
- [ ] Dashboard charts and counts use real API responses.
- [ ] Ask Data supports question -> interpretation -> SQL preview -> validation -> explicit execution -> results.
- [ ] Ambiguous/unsupported requests do not yield executable SQL.
- [ ] Unsafe statements and non-allowlisted tables are rejected.
- [ ] The database reader role cannot write to business data.
- [ ] Results are bounded by row, time, and output-size limits.
- [ ] Data-quality report shows reproducible rules/findings.
- [ ] Anomaly endpoint uses a real fitted model and reports score, version, and limitations.
- [ ] Benchmark runner executes at least 30 reviewed cases and stores actual metrics.
- [ ] Demo mode is visibly labelled; provider-connected mode is visibly labelled.
- [ ] Docker Compose starts all required services.
- [ ] Backend tests pass and frontend type/build checks pass.
- [ ] No real secrets are committed; .env is ignored.
- [ ] README, architecture, API, security, evaluation, and demo docs exist.
- [ ] Known limitations are written down; no fake claims are made.

Reliability targets

Use these as goals, not guaranteed outcomes:

- Zero forbidden operations execute in the security test suite.
- All core user flows pass the manual smoke checklist.
- Benchmark results are reported for the actual cases run, with the mode/provider identified.
- For answerable benchmark cases, aim for at least 80% execution correctness on the curated supported subset; if not achieved, report the actual result and analyze failures.
- For the multi-turn subset, aim for at least 80% correct behavior; do not conceal failures or quietly remove difficult cases.

22. Seven-day build plan

Day 1 - foundation

- Inspect workspace, create implementation plan, repo structure and BUILD_STATUS.md.
- Create FastAPI app and React app.
- Create database schema/migrations and deterministic synthetic seed generator.
- Add health/readiness endpoints, .env.example, and first tests.
- At end of day: app launches, DB seeds and dashboard endpoint returns real counts.

Day 2 - frontend/backend skeleton and auth

- Build global shell, login, dashboard cards/charts and device table.
- Implement password hashing, access token and RBAC dependencies.
- Add typed frontend API client and clear loading/error/empty states.
- Verify Viewer/Analyst/Admin restrictions.

Day 3 - natural-language preview

- Build provider abstraction, prompt and strict JSON schema.
- Implement demo mode and one real provider if credentials are available.
- Add semantic catalog and conversation storage.
- Implement SQL parsing/validation and preview endpoint.
- At end of day: supported questions show a validated preview, ambiguous questions ask for clarification.

Day 4 - safe execution

- Create separate read-only database role.
- Implement execute-by-preview-ID flow, row/time/output limits and query history.
- Test blocked SQL classes and ownership/RBAC.
- At end of day: safe query runs and unsafe query is blocked.

Day 5 - data quality and ML

- Implement profiling rules and Data Quality screen.
- Implement IsolationForest telemetry pipeline, model metadata and Anomaly Monitor screen.
- Verify findings and anomaly scores against controlled seeded data.

Day 6 - benchmark and hardening

- Add 30+ benchmark cases with expected behavior/results.
- Run baseline versus guarded pipeline where applicable.
- Add regression report, rate limiting/cache only if core functions are complete.
- Run complete tests and fix discovered failures.

Day 7 - containerization and presentation

- Finalize Dockerfiles/Compose and local start instructions.
- Test clean startup, persistence, roles and all key flows.
- Refine UI responsiveness and accessibility.
- Prepare README, architecture/API/security docs, demo script, limitations, and actual evaluation report.
- Add Kubernetes manifests only if time and course evaluation require it.

If time runs short, preserve this order: safe end-to-end SQL path, dashboard, tests, data quality, ML component, benchmark UI, deployment polish, stretch items. Do not sacrifice SQL safety to add more features.

23. Suggested live demo (5-7 minutes)

1. **20 seconds:** Introduce QueryTrust and state that the data is synthetic.
2. **40 seconds:** Show dashboard counts and device/warranty visuals from PostgreSQL.
3. **60 seconds:** Ask “How many active routers are in the North region?” Preview SQL and validation findings before clicking Execute.
4. **45 seconds:** Ask an ambiguous question such as “Show high-risk devices” and demonstrate clarification instead of a guessed definition.
5. **45 seconds:** Demonstrate that a forbidden write query is rejected; explain the read-only DB role and parser checks.
6. **45 seconds:** Run the data-quality profiler and show specific missing/inconsistent fields.
7. **45 seconds:** Run anomaly scan and explain that the scores are synthetic-data outlier signals, not failure probabilities.
8. **60 seconds:** Run the benchmark page and explain the real measured results, including one failure and how it informed a regression test.
9. **30 seconds:** Show the architecture, Docker Compose, test status and limitations.

Do not claim results that were not actually demonstrated. If an external model API is unavailable during the viva, use demo mode and clearly identify it as a deterministic fallback; do not call it a live LLM run.

24. Viva preparation: understand these decisions

The student should be able to explain, in their own words:

- Why a full database is not put into an LLM context and why SQL remains the execution engine for structured aggregation.
- Difference between natural-language intent, SQL generation, SQL validation, SQL execution, and result explanation.
- Why prompts are not a security boundary.
- Why a read-only DB role is essential even when SQL is parsed.
- Why SQL text equality is not the same as result correctness.
- How the semantic catalog defines business terms consistently.
- How multi-turn state can cause errors and why follow-ups need regression tests.
- Why missing value imputation is a recommendation, not an automatic truth; never infer sensitive attributes without justification.
- Why IsolationForest identifies outliers but does not estimate probability of device failure.
- How an LLM provider failure is handled.
- How JWT/RBAC and backend dependency checks work.
- How Docker Compose connects the services and how environment variables keep secrets outside source control.
- Which portions are implemented, which are stretch work, and what the measured evaluation actually shows.

25. Required documentation and hand-in package

The code workspace should contain:

1. README.md - complete setup, configuration, demo users, provider setup, tests and troubleshooting.
2. PROJECT_SPEC.md - this specification.
3. ARCHITECTURE.md - components, flow diagram, design choices.
4. API_CONTRACT.md - routes, request/response schema and auth requirements.
5. SECURITY.md - threat boundaries, SQL guard, database roles, known limitations.
6. EVALUATION_PLAN.md - benchmark design, metric definitions, data version, baseline and guarded comparison.
7. DEMO_SCRIPT.md - ordered presentation steps and fallback plan.
8. BUILD_STATUS.md - what is implemented, what is tested, known failures, next work.

9. .env.example and .gitignore.
10. Automated tests and benchmark cases.
11. Dockerfiles and Compose configuration.
12. A generated evaluation report containing actual values from a run.

The final report must distinguish planned behavior, implemented behavior, tested behavior, and unverified behavior.

26. Antigravity implementation protocol

The agent must use an iterative engineering workflow:

1. Read this entire spec and AGENTS.md before coding.
2. Inspect the actual workspace, available runtime versions, existing files and credentials/configuration status. Never delete existing work without asking.
3. Create docs/BUILD_STATUS.md and a milestone checklist with file-level tasks and test commands.
4. Start with the smallest vertical slice: DB seed -> FastAPI endpoint -> React data display.
5. Implement one milestone at a time; run its tests before advancing.
6. Use actual terminal output to decide whether a test/build passes. Never fabricate success.
7. Use the browser agent or a browser test to inspect the rendered application if available. Fix layout defects, broken routes and console errors.
8. Keep the code idiomatic, modular and documented. Avoid monolithic files and over-engineering.
9. If a third-party service/API key is absent, implement and label demo mode rather than blocking the entire project.
10. If requirements conflict, prioritize: data safety, end-to-end correctness, course requirements, then optional polish.
11. At the end of each milestone, summarize files changed, commands run, test results, limitations and next milestone.
12. At completion, report only verified facts and provide exact run/test commands.

Do not write the entire application in one uncontrolled generation pass. Do not stop after creating a scaffold. Continue through implementation, tests, fixing failures and documentation until the acceptance criteria are met or a real blocker is clearly reported.

27. Course alignment checklist

Course learning outcome	Project evidence	Required artifact/demo
CLO01 - REST APIs and SQL/NoSQL database design	Versioned REST routes, relational schema, migrations, indexes, pagination	OpenAPI docs, schema diagram, API tests
CLO02 - authentication/authorization, scalable/performance-conscious backend	Password hashes, signed tokens, RBAC, separate read-only DB role, query timeouts/limits, optional cache/rate limits	Role matrix, blocked-query demo, tests
CLO03 - modern responsive frontend and interactive dashboards	React/TypeScript pages, component reuse, data fetching, charts, loading/error states	Browser demo, build/type checks
CLO04 - model serving and containerized deployment	FastAPI anomaly inference endpoint and Docker images/Compose	Model endpoint test, Docker Compose demo
CLO05 - model versioning, monitoring and safe updates	Model metadata, structured logs, query/prompt versions, evaluation runs and regression benchmark	Evaluation report, version display, operational notes

The course handout lists Docker/Kubernetes and ML monitoring/versioning in its scope. Docker Compose and explicit model metadata are the MVP baseline; Kubernetes/MLflow can be added if required and time permits. Confirm any project-specific compulsory deployment requirement with the course instructor.

28. Risks and trade-offs

Risk	Mitigation
No LLM API key	Demo mode remains functional; clearly label the mode; implement provider interface
Invalid or unsafe generated SQL	Parser-based guard, allowlist, read-only DB user, statement timeout, row cap, tests
Multi-turn context drift	Bounded session history, semantic catalog, explicit clarification, regression cases
LLM appears correct but metric definition is wrong	Define business semantics; show interpreted question and assumptions; verify results
Seven-day scope creep	P0/P1/stretch order; stop adding features until core smoke test passes
Fake-looking dashboard	All finished dashboard values come from the database/API; no hard-coded production metrics
Model misrepresented as predictive failure AI	Label as outlier detection, expose model version and limitations
Data privacy	Synthetic data only; do not connect real company or Cisco datasets; mask logs/secrets
Prompt/provider change breaks old cases	Version prompts, benchmark suite, regression report
Deployment claims exceed evidence	Report exact commands actually tested and whether Kubernetes was run

29. Reference links for implementation

Use primary documentation first and verify current APIs/version behavior during implementation.

- Google Antigravity rules and persistent workspace instructions: https://www.antigravity.google/docs/rules/
- Google Antigravity IDE overview: https://www.antigravity.google/docs/ide/overview/
- Snowflake Cortex Analyst evaluation documentation: https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst-evaluations
- ThoughtSpot Spotter product overview: https://www.thoughtspot.com/product/agents/spotter
- FastAPI: https://fastapi.tiangolo.com/
- React: https://react.dev/
- PostgreSQL: https://www.postgresql.org/docs/
- SQLGlot: https://sqlglot.com/
- SQLAlchemy: https://docs.sqlalchemy.org/
- Alembic: https://alembic.sqlalchemy.org/
- Docker Compose: https://docs.docker.com/compose/
- scikit-learn IsolationForest: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html
- pytest: https://docs.pytest.org/

30. Final build principle

Build a small system whose most important path works end-to-end and whose limitations are visible. A trustworthy demonstration is better than a wide but fragile feature list. Prefer measurable correctness, guarded database execution, useful interaction design, reproducible tests, and honest documentation over unsupported claims of enterprise scale or originality.